



Bayesian Filtering Approaches for 3D Localization of Quadruped Robots on the Grand Tour Dataset

Hao Duan, Xiuping Yang, Zexing Lin, Junxian Wang
University of Michigan, Ann Arbor, MI 48105



Introduction

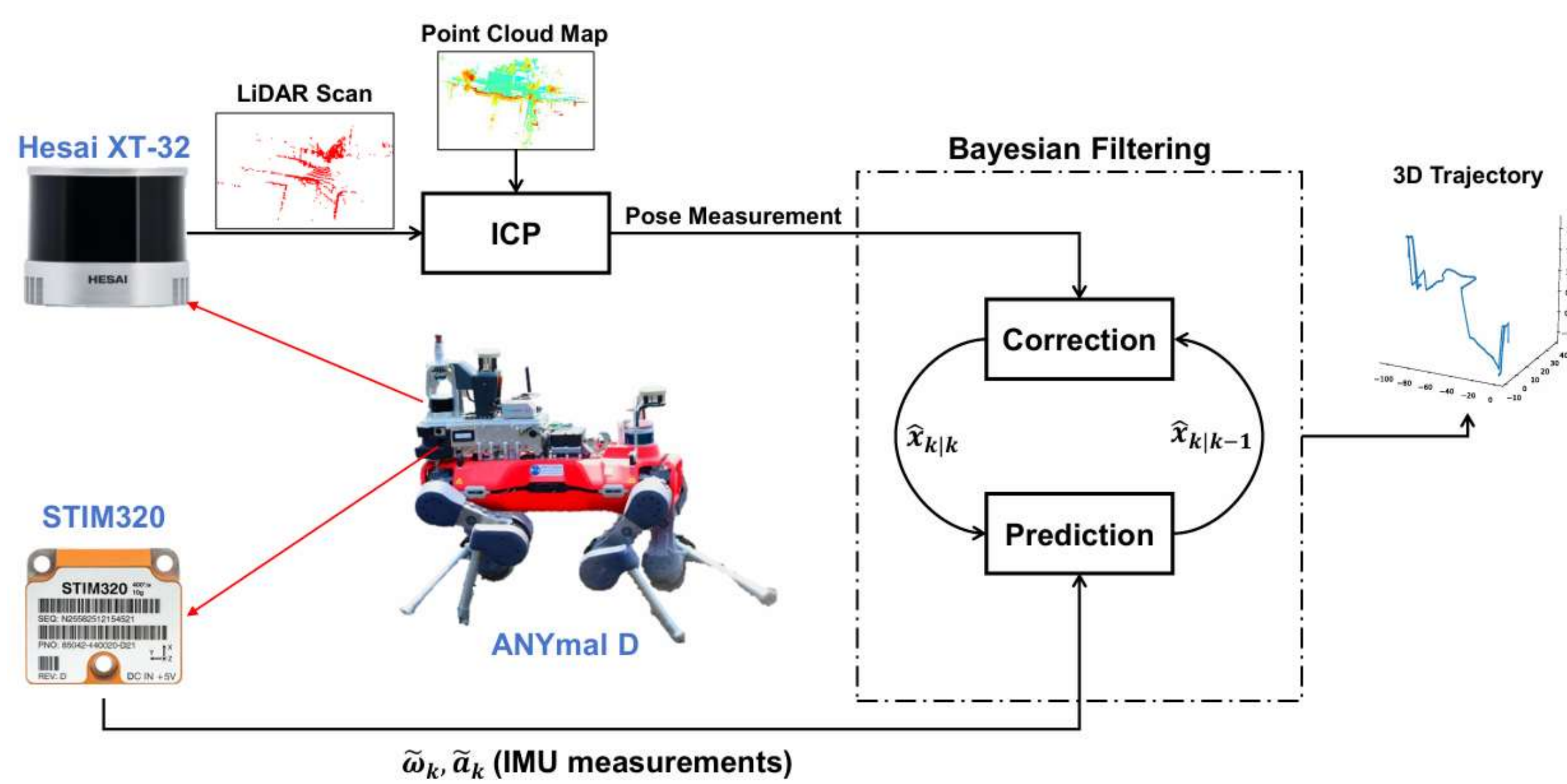
Motivation

- Accurate state estimation is a cornerstone challenge for legged platforms operating in complex environments due to terrain irregularities, foot slipping, and sensor noise.
- Need for a systematic evaluation of state estimation algorithms to determine the most robust strategy for specific constraints and environmental conditions.

Objectives

- Conduct a comprehensive benchmark of mainstream Bayesian filtering frameworks using the ANYmal GrandTour Dataset.
- Deeply investigate the impact of different approximation and sampling strategies on 3D localization performance in real-world scenarios.

Methodology



1 IMU-LiDAR State Estimation via Bayesian Filtering

- Frameworks:** Build the map, and then fuse IMU prediction with LiDAR ICP scan-to-map correction and compare four frameworks—EKF (first-order linearization), UKF (sigma-point nonlinear handling), PF (non-Gaussian, multi-modal), and InEKF (Lie-group-consistent error dynamics).

2

Algorithm Implementation

Algorithm 1 PF-Based Radar-IMU Trajectory Fusion
Require: Map $\mathcal{M}$, IMU $\mathcal{S}_{imu}$, LiDAR $\mathcal{S}_{lidar}$, Estimator $\mathcal{T}_E$.
Ensure: Trajectory $\mathcal{T}$.
1: Init: Load map $\mathcal{M}$; Align 1st scan via ICP; Init particles $\{x_i, w_i\}_{i=1}^N$.
2: while New LiDAR frame P_t arrives do
3: 1. Prediction (IMU Propagation):
4: for each particle i do
5: Integrate IMU $\mathcal{S}_{imu}$ to update (R_i, v_i, p_i) ;
6: end for
7: 2. Multi-Candidate ICP:
8: Select K indices $\mathcal{S}$ (Best, Mean, Random);
9: for each $j \in \mathcal{S}$ do
10: $T_{ij} \leftarrow T_{ij}^{(j)}$; $T_{ij} \leftarrow T_{ij}^{(j)}$;
11: $T_{ij} \leftarrow \text{ICP}(P_t, \mathcal{M}, T_{ij})$;
12: $s_j \leftarrow \text{Fitness} - \lambda \cdot \text{RMSE}$;
13: end for
14: $(j^*, T^*) \leftarrow \text{argmax}_j(s_j)$;
15: for each particle i do
16: if $\text{Fitness}(T^*) > \tau_j$ and $\text{RMSE}(T^*) \leq \tau_r$ then
17: Update weight $w_i \propto w_i \cdot \mathcal{P}(s_i; T^*, \Sigma)$;
18: end if
19: end for
20: Normalize w_i ;
21: end if
22: 3. Estimation & Resample:
23: Estimate $\hat{x} \leftarrow \sum w_i x_i$;
24: if $N_{eff} < N_{thr}$ then
25: Resample $\{x_i\}$; Apply Roughening;
26: end if
27: Store $\hat{x}$ in $\mathcal{T}$;
28: end while
29: return $\mathcal{T}$

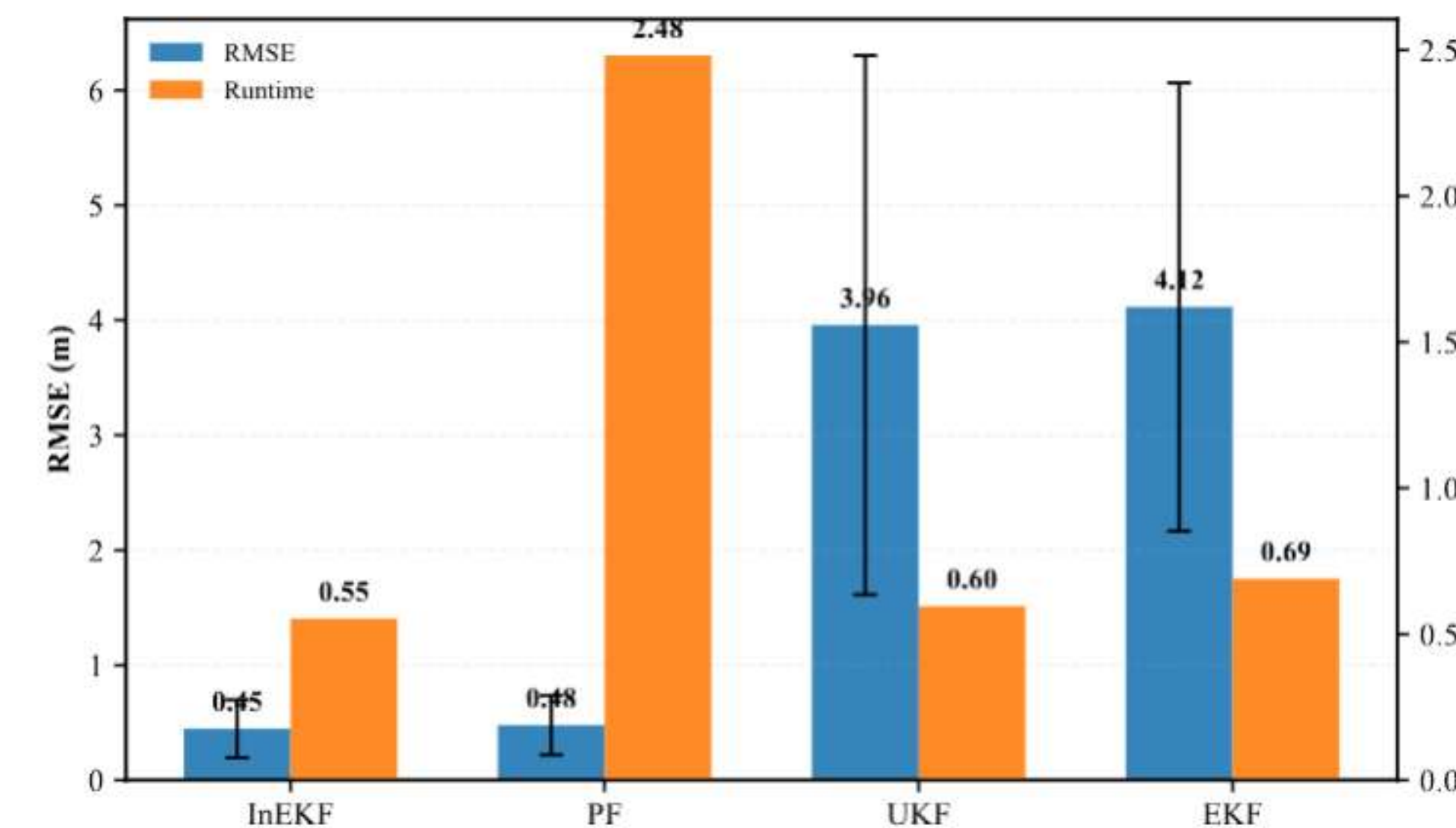
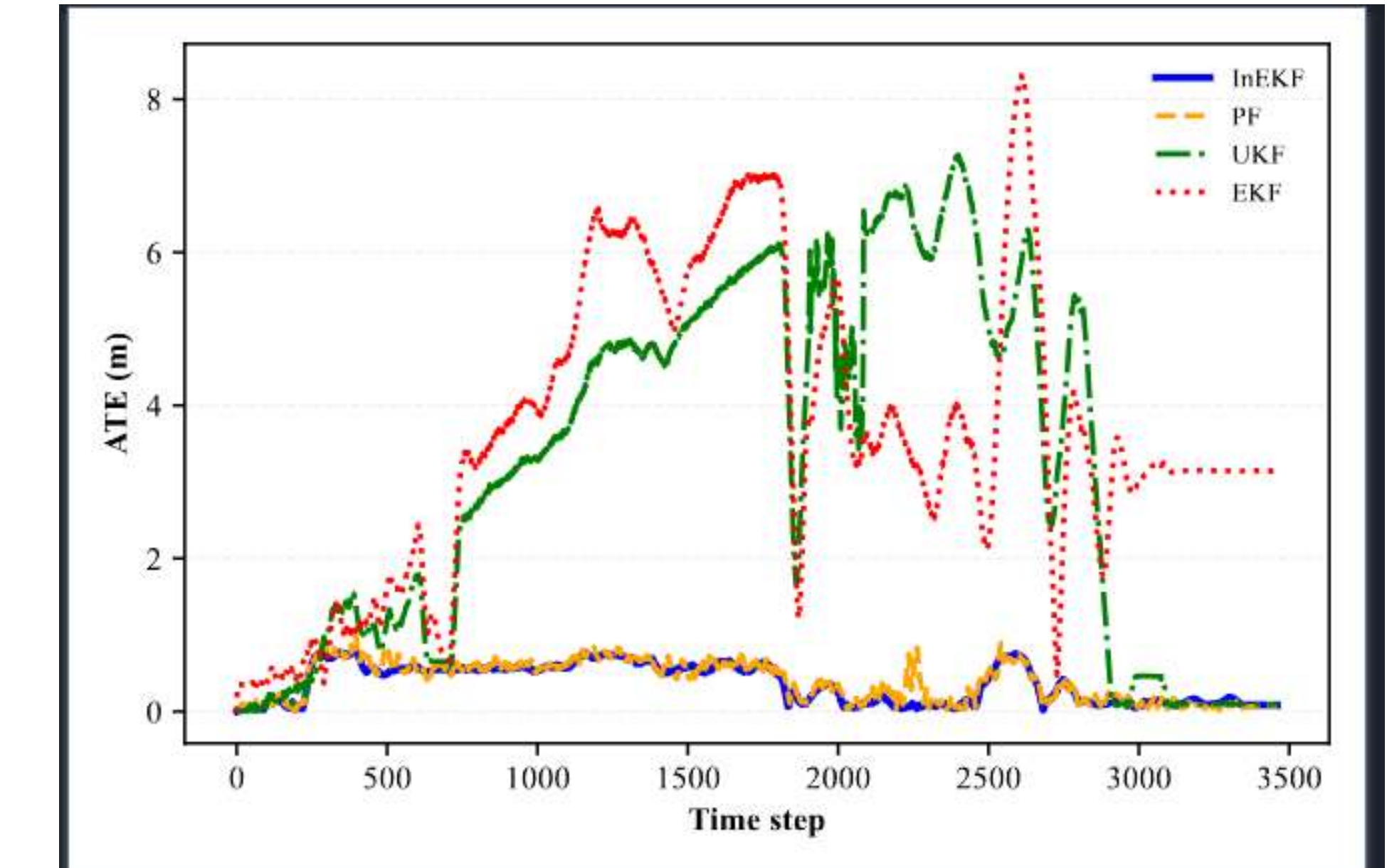
Algorithm 1 UKF-Based Radar-IMU Trajectory Fusion
Require: previous state mean μ_{k-1} , previous covariance Σ_{k-1} , IMU angular velocity ω_k , IMU linear acceleration a_k , radar position measurement z_k .
1: Estimate gyro bias, initial gravity direction, and residual acceleration bias from the initial IMU segment
2: Integrate ω_k to update the sensor orientation
3: Transform a_k from the body frame to the world frame and remove gravity
4: $\hat{x}_{k-1} \leftarrow$ generate sigma points from μ_{k-1} and Σ_{k-1}
5: $w \leftarrow$ compute the corresponding sigma-point weights
6: for each sigma point $\hat{x}_{k-1}^{(i)}$ do
7: $s_i^{(j)} \leftarrow f(\hat{x}_{k-1}^{(j)}, \hat{a}_k, \Delta t)$
8: end for
9: $\hat{\mu}_k \leftarrow \sum_{i=1}^n w_i s_i^{(j)}$
10: $\hat{\Sigma}_k \leftarrow \sum_{i=1}^n w_i (s_i^{(j)} - \hat{\mu}_k)(s_i^{(j)} - \hat{\mu}_k)^T + Q_k$
11: Transform the radar ICP trajectory into the IMU frame using the static extrinsic calibration
12: $\hat{x}_k \leftarrow$ generate sigma points from $\hat{\mu}_k$ and $\hat{\Sigma}_k$
13: $w \leftarrow$ compute the corresponding sigma-point weights
14: for each predicted sigma point $\hat{x}_k^{(i)}$ do
15: $z_i^{(j)} \leftarrow h(\hat{x}_k^{(j)})$
16: end for
17: end for
18: $\hat{\Sigma}_k \leftarrow \sum_{i=1}^n w_i (z_i^{(j)} - \hat{z}_k)(z_i^{(j)} - \hat{z}_k)^T + R_k$
19: $\hat{\mu}_k \leftarrow \hat{\mu}_k + K_k R_k^{-1}$
20: $\hat{\Sigma}_k \leftarrow \hat{\Sigma}_k - K_k S_k K_k^T$
21: $\hat{\mu}_k \leftarrow \hat{\mu}_k + K_k S_k K_k^T$
22: return $\hat{\mu}_k, \hat{\Sigma}_k$

Algorithm 1 EKF Algorithm
Require: previous state mean μ_{k-1} , previous covariance Σ_{k-1} , control input u_k , measurement z_k , process noise covariance Q_k , measurement noise covariance R_k , nonlinear state transition $f(\cdot)$, nonlinear measurement $h(\cdot)$.
1: Predict the state mean
2: $\hat{\mu}_k \leftarrow f(\mu_{k-1}, u_k)$
3: Compute the state transition Jacobian
4: $F_k \leftarrow \frac{\partial f(\mu_{k-1})}{\partial \mu_{k-1}}|_{\mu_{k-1}=\hat{\mu}_{k-1}}$
5: Predict the covariance
6: $\hat{\Sigma}_k \leftarrow F_k \Sigma_{k-1} F_k^T + Q_k$
7: // Update (Measurement Update)
8: Predict the measurement
9: $\hat{z}_k \leftarrow h(\hat{\mu}_k)$
10: Compute the measurement Jacobian
11: $H_k \leftarrow \frac{\partial h(\hat{\mu}_k)}{\partial \mu_k}|_{\mu_k=\hat{\mu}_k}$
12: Compute the innovation (residual)
13: $y_k \leftarrow z_k - \hat{z}_k$
14: Compute the innovation covariance
15: $S_k \leftarrow H_k \hat{\Sigma}_k H_k^T + R_k$
16: Compute the Kalman gain
17: $K_k \leftarrow \hat{\Sigma}_k H_k^T S_k^{-1}$
18: Update the state mean
19: $\hat{\mu}_k \leftarrow \hat{\mu}_k + K_k y_k$
20: Update the covariance
21: $\hat{\Sigma}_k \leftarrow (\hat{\Sigma}_k - K_k H_k) S_k^{-1} (I - K_k H_k)^T + K_k R_k K_k^T$
22: return $\hat{\mu}_k, \hat{\Sigma}_k$

Algorithm 1 RIEKF (Right-Invariant Error EKF)
Require: previous estimate $R_{k-1}, p_{k-1}, v_{k-1}, b_{k-1}, g_{k-1}$, previous covariance P_{k-1} , IMU angular velocity ω_k , IMU linear acceleration a_k , measurement z_k (e.g., radar position), time step Δt .
1: Estimate gyro bias, initial gravity direction, and residual acceleration bias from the initial IMU segment
2: $\hat{\omega}_k \leftarrow \omega_k - b_{\omega,k-1}$; $\hat{a}_k \leftarrow a_k - b_{a,k-1}$
3: $R_k \leftarrow R_{k-1} \exp(\hat{\omega}_k^* \Delta t)$
4: $v_k^* \leftarrow R_k \hat{a}_k - g$
5: $p_k \leftarrow p_{k-1} + v_k^* \Delta t + \frac{1}{2} \hat{a}_k^* \Delta t^2$
6: $\hat{p}_k \leftarrow p_{k-1} + v_{k-1} \Delta t + \frac{1}{2} \hat{a}_{k-1}^* \Delta t^2$
7: $\Phi_k, Q_k \leftarrow$ from RIEKF error dynamics
8: $P_k \leftarrow \Phi_k P_{k-1} \Phi_k^T + Q_k$
9: $P_k^* \leftarrow R_k P_k + P_k$
10: $\hat{z}_k \leftarrow z_k - \hat{p}_k^*$
11: $H_k \leftarrow \frac{\partial \hat{h}(\hat{p}_k^*)}{\partial p_k}|_{p_k=\hat{p}_k^*}$
12: $S_k \leftarrow H_k P_k H_k^T + R_k$
13: $K_k \leftarrow P_k H_k^T S_k^{-1}$
14: $\hat{p}_k^* \leftarrow \hat{p}_k^* + K_k \hat{z}_k$
15: $R_k \leftarrow R_k - K_k S_k K_k^T$
16: $p_k \leftarrow p_k + \hat{p}_k^*$
17: $v_k \leftarrow v_k + \hat{a}_k^*$
18: $b_{\omega,k} \leftarrow b_{\omega,k-1} + \delta b_{\omega,k}$
19: $b_{a,k} \leftarrow b_{a,k-1} + \delta b_{a,k}$
20: $P_k \leftarrow (I - K_k H_k) P_k (I - K_k H_k)^T + K_k R_k K_k^T$
21: return $R_k, p_k, v_k, b_{\omega,k}, b_{a,k}, P_k$

2

Error and Runtime comparison



Discussion

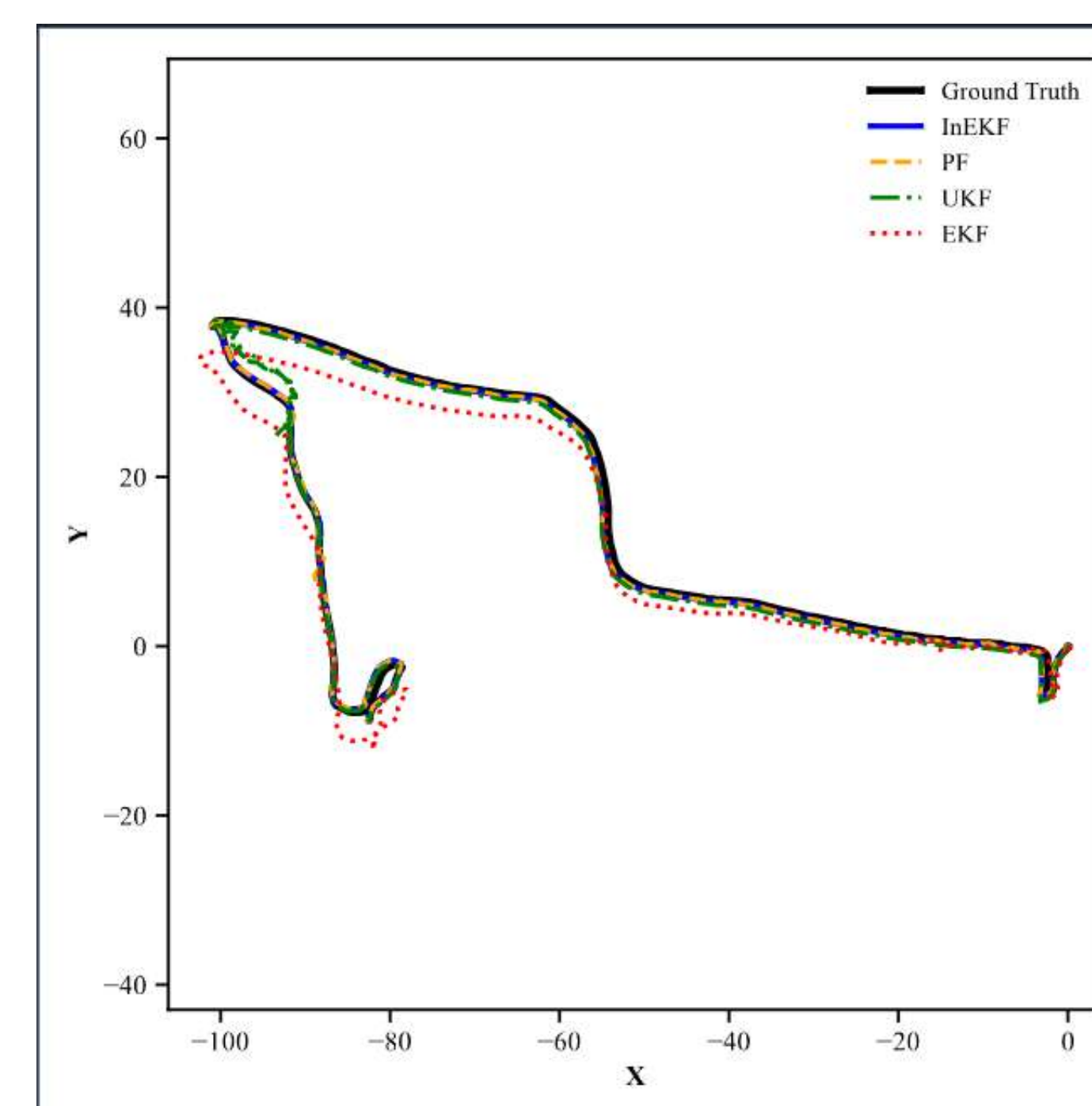
- Our approach achieves high performance with real-time capability, especially regarding the In-EKF filter.
- All methods perform similarly in simple environments, but InEKF shows better robustness in challenging scenarios, while PF is more computationally expensive.
- The main limitation is the reliance on ICP measurements, which can lead to drift when the measurements become unreliable.

Future Works

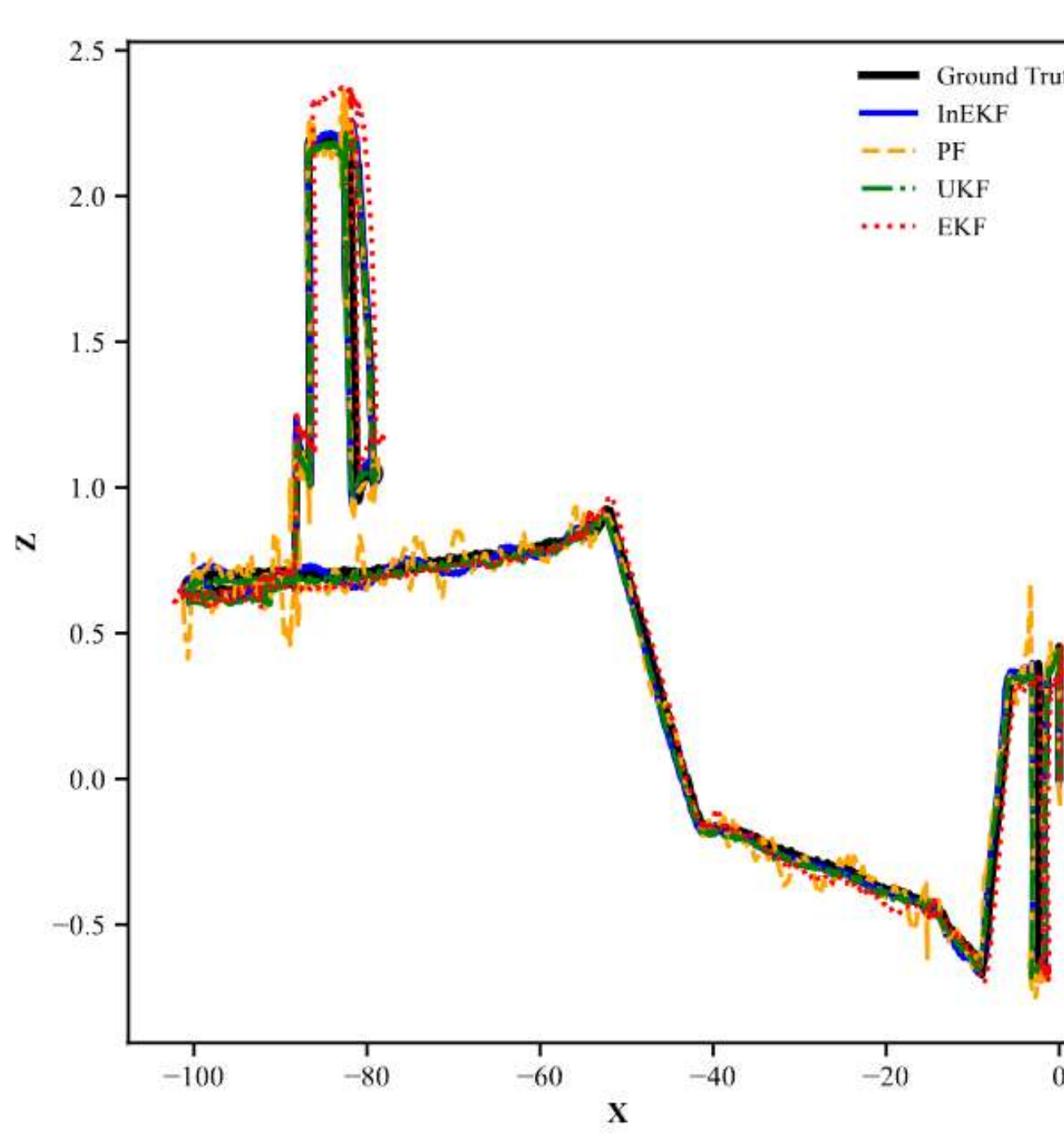
- Future work will focus on improving robustness under perceptual degeneracy.
- Another promising direction is to incorporate contact information within an invariant filtering framework.

1

Trajectory Comparison



Top view



Side view

References: [1] J. Frey, T. Tuna, F. Fu, K. Patterson, T. Xu, M. Fallon, C. Cadena, and M. Hutter, "GrandTour: A Legged Robotics Dataset in the Wild for Multi-Modal Perception and State Estimation," arXiv:2602.18164, 2026.

[2] Kalman, Rudolph Emil. "A new approach to linear filtering and prediction problems." (1960): 35-45.

[3] Julier, Simon J., and Jeffrey K. Uhlmann. "New extension of the Kalman filter to nonlinear systems." Signal processing, sensor fusion, and target recognition VI. Vol. 3068. Spie, 1997.

[4] Smith, Adrian. Sequential Monte Carlo methods in practice. Springer Science & Business Media, 2013.

[5] Barrau, Axel, and Sil v re Bonnabel. "The invariant extended Kalman filter as a stable observer." IEEE Transactions on Automatic Control 62.4 (2016): 1797-1812.

[6] Thrun, Sebastian. "Probabilistic robotics." Communications of the ACM 45.3 (2002): 52-57.

Acknowledgment: We would like to thank Prof. Maani Ghaffari, Dr. Minghan Zhu, and the GSIs for their inspiring lectures, helpful instruction, and support during the semester and throughout this project.